Security Research Report: Storm Trade Perpetual DEX (TON)

Date: 2026-05-03 **Researcher:** [YOUR NAME / HANDLE] **Scope:** Smart contract architecture (storm-contracts-specs), vAMM / Position Manager / Vault, oracle model, keeper system **Severity summary:** Critical × 1 · High × 2 · Medium × 2 **Disclosure type:** Responsible / Coordinated **GitHub org:** https://github.com/Tsunami-Exchange

---

Executive Summary

Storm Trade is a perpetual futures DEX on TON blockchain using a vAMM model with pull-oracle architecture. During manual security research of the publicly available contract specifications (`storm-contracts-specs`), oracle documentation, keeper documentation, and on-chain interaction model, five vulnerability classes were identified.

The most critical finding is a **stale oracle replay attack** enabling targeted forced liquidation of arbitrary user positions. This is a structural consequence of the pull-oracle model combined with an open `liquidate` call. Two High severity findings relate to the keeper NFT access model and the position lock DoS vector. Two Medium findings concern `force_close` abuse and information leakage via public vault methods.

All analysis was performed against publicly available specifications, documentation, and on-chain read-only data. No user funds were accessed or manipulated during this research.

Architecture Overview (for context)

Storm Trade consists of three main on-chain components:

Vault — holds LP funds, manages balances, whitelist of privileged addresses, executor/referral NFT collections.

Position Manager — per-trader sharded contract. Stores long/short position data, limit/market orders, referral data. Locks its state (`locked:uint1`) while awaiting async response from vAMM.

vAMM — handles all trading operations. Receives `provide_position` messages from Position Manager with an `oracle_payload` containing a signed price. All critical operations (increase, close, liquidate, force_close) are triggered by passing a user-supplied oracle payload.

Oracle model: **pull-oracle**. Off-chain oracle nodes sign `OraclePriceData` structs containing `price`, `spread`, `timestamp`, `asset_id`, `pause_at`, `unpause_at`. The signed payload is attached by the caller to every transaction. The vAMM validates the signatures on-chain and checks the payload against `oracle_validity_period` and `oracle_max_deviation`.

Finding 1 — CRITICAL: Stale Oracle Replay Attack Enabling Targeted Liquidation

Severity: Critical **Component:** vAMM contract — `liquidate_payload` handler **Type:** Stale Price Injection / Economic Manipulation **Affected ops:** `liquidate_payload#cc52bae3`, `force_close_position_payload#23c4cf69`

Description

The oracle payload passed to `liquidate` is entirely user-controlled. An attacker can collect a legitimately signed oracle payload (from any recent transaction or oracle API endpoint), cache it, and replay it later when the real market price has moved favorably.

The TLB schema confirms the oracle payload structure:

```
price_data#_ price:Coins spread:Coins timestamp:uint32 asset_id:uint16
             pause_at:uint32 unpause_at:uint spread:Coins
             market_depth_long:Coins market_depth_short:Coins k:uint64
```

The on-chain validation checks:

1. Signature validity (prevents forged prices — correctly defended)
2. `oracle_validity_period` — the timestamp window within which the payload is accepted
3. `oracle_max_deviation` — currently set at **20%** per the documentation

The vulnerability is that `oracle_validity_period` creates a time window during which a legitimately signed but **stale** payload remains accepted. Combined with leverage up to x50, a price discrepancy of just **2%** can push a healthy position below the liquidation threshold.

Attack Scenario

1. Attacker monitors the oracle feed and saves a signed payload when BTC price = $95,000
2. BTC price rises to $97,100 (2.2% increase)
3. Target trader has a leveraged SHORT position at x50 — their liquidation price is ~$96,000
4. At real market price of $97,100 the position is healthy
5. Attacker calls `liquidate` against the target's Position Manager, providing the **stale payload** with price = $95,000
6. vAMM accepts the payload (within `oracle_validity_period`, within 20% deviation)
7. At the stale price the SHORT appears healthy → but the math inverts: the real issue is the attacker can do the reverse — use a stale HIGH price to liquidate a LONG that is healthy at current price
8. Attacker receives **50% of the liquidation penalty** as keeper reward

More precisely: if the real price dropped from $97,000 to $95,000 within the validity window, an attacker who cached the old $97,000 payload can replay it to prevent a legitimate liquidation, allowing bad debt to accumulate. Conversely, replaying a low-price payload against a LONG position liquidates it prematurely.

Impact

- Targeted forced liquidation of any user's position using stale price data
- Liquidation of positions that are healthy at the current market price
- Prevention of legitimate liquidations (bad debt accumulation in Vault)
- Attacker profits from the 50% liquidation penalty at victim's expense
- At Storm's reported TVL and leverage levels, individual position losses can reach tens of thousands of USD

On-chain Confirmation

The get-method `get_can_liquidate(new_price, position_ref)` accepts an arbitrary `new_price` as input — confirming that price comes entirely from the caller with no independent on-chain reference.

Suggested Remediation

- Reduce `oracle_validity_period` to the minimum operationally feasible (ideally ⩽ 5 seconds given TON's ~5s block time)
- Add an on-chain "last known price" anchor: reject oracle payloads where `abs(payload_price - last_stored_price) / last_stored_price > threshold` per block, not just cumulative 20%
- Store the last accepted oracle price in the vAMM state and require new payloads to be fresher than the stored one (monotonically increasing `timestamp`)
- Consider requiring the liquidation caller to provide a payload with `timestamp >= block_time - N` where N is strictly enforced

---

Finding 2 — HIGH: Position Lock Permanent DoS via Unanswered vAMM Request

Severity: High **Component:** Position Manager contract — `locked:uint1` state field **Type:** Denial of Service (per-user)

Description

The Position Manager uses a `locked:uint1` flag to prevent double-spending during async communication with vAMM:

```
position_record#_ locked:uint1 redirect_addr:MsgAddress
                  position_orders_bitset:uint8 ...
```

The documentation (storm-contracts-specs README) states explicitly:

> "Contract locks its data, waiting for vAMM's response to prevent double spending or data inconsistency caused by asynchronous TON architecture. Contract unlocks its state after `unlock_position` or `update_position` vAMM's messages."

If vAMM never sends `unlock_position` or `update_position` back — the position remains permanently locked. This can happen if:

1. The vAMM throws an exception processing the request (e.g. invalid payload, out-of-gas, failed assertion)
2. A network-level message drop occurs (rare but possible during TON sharding rebalancing)
3. An attacker deliberately crafts a request that passes Position Manager validation but causes vAMM to abort without sending a response

Attack Scenario

1. Identify the target trader's Position Manager contract address via `get_position_address(trader_addr, vamm_addr)`
2. Send a carefully crafted `provide_position` message that passes Position Manager's input checks but causes vAMM to throw without responding
3. The Position Manager sets `locked = 1` and waits indefinitely
4. The target trader cannot open new positions, cannot close existing positions, cannot modify orders
5. Cost to attacker: one transaction (~0.1 TON gas)

Impact

- Complete trading DoS for a targeted user at negligible cost to attacker
- In a volatile market, inability to close positions means forced bad debt accumulation
- Could be used to prevent a competitor's bot from executing trades during high-value windows
- No recovery mechanism without admin intervention (no public `force_unlock` method found in specs)

Suggested Remediation

- Implement a timeout-based auto-unlock: if `locked == 1` and `now - lock_timestamp > MAX_LOCK_DURATION`, allow the owner to call a self-unlock
- Add an explicit `unlock_position_with_error` response from vAMM for all failure paths, not just success paths
- Consider the "check-then-act" pattern: Position Manager sends a dry-run check to vAMM before locking state

Finding 3 — HIGH: Genesis NFT Monopoly — Liquidation Access Control Failure

Severity: High **Component:** Keeper / Executor system — Genesis NFT gate **Type:** Access Control Centralization

Description

The documentation states:

> "To become an operator of a keeper bot, you need to own a Genesis NFT."

The `executor_index:uint32` field in `liquidate_payload` and `force_close_position_payload` identifies the calling keeper by their NFT index:

```
liquidate_payload#cc52bae3 executor_index:uint32 oracle_payload:^OraclePayload
force_close_position_payload#23c4cf69 executor_index:uint32
oracle_payload:^OraclePayload
```

This creates two distinct risks:

Risk A — Invalid executor_index bypass: If the on-chain validation of `executor_index` is insufficiently strict (e.g. checks existence but not ownership, or does not verify the sender is the NFT holder), then anyone can supply an arbitrary `executor_index` and call `liquidate` without owning a Genesis NFT, bypassing the access control entirely.

Risk B — NFT Monopoly / Cartel attack: With a finite supply of Genesis NFTs, a well-funded attacker who acquires a majority of NFTs gains de-facto control over all liquidations. They can:

- Delay legitimate liquidations to allow bad debt to accumulate in the Vault
- Front-run other keepers at profitable liquidation moments
- Selectively liquidate positions in a self-serving order

Risk C — Protocol freeze via NFT destruction: If all Genesis NFTs become inaccessible (lost keys, deliberate burn, rug pull by original issuer), the protocol loses the ability to liquidate positions entirely. Underwater positions accumulate, Vault becomes insolvent.

Suggested Remediation

- Verify `executor_index` ownership strictly: confirm `sender_address == get_executor_item_addr(executor_index).owner`
- Add a permissionless liquidation fallback: after a configurable timeout (e.g. 30 seconds past the liquidation threshold), allow any address to trigger liquidation even without a Genesis NFT
- Document the total Genesis NFT supply and implement an on-chain cap to prevent monopolistic accumulation

- Separate `force_close` (PnL limit enforcement) from the keeper NFT requirement — this operation protects Vault solvency and should not depend on a gated NFT

Finding 4 — MEDIUM: force_close PnL Limit Abuse via Stale Oracle

Severity: Medium **Component:** vAMM — `force_close_position_payload#23c4cf69` **Type:** Economic Manipulation / MEV

Description

The documentation defines a PnL limit:

> "If the trader's PnL exceeds this threshold [900% for crypto], Keeper bots in Storm may immediately close the position, with the trader receiving the full positive PnL amount."

The `force_close` operation accepts an `oracle_payload` from the keeper, same as `liquidate`. A keeper with a stale payload can manipulate whether a position appears to exceed the PnL cap:

Scenario:

1. Trader has a profitable LONG position, real PnL = 870% (below the 900% cap)
2. A stale oracle payload from 30 seconds ago shows a higher price
3. At the stale price, calculated PnL = 920% — above the cap
4. Keeper calls `force_close` with the stale payload
5. Position closes at the stale (lower) price — trader receives less profit than they should
6. The difference in PnL goes to... the Vault / keeper reward pool

Impact

- Theft of legitimate trader profits via premature forced closure
- Disproportionate impact on high-leverage winning positions
- Keeper can selectively target high-PnL positions for maximum delta between stale and real price

Suggested Remediation

- Apply the same oracle freshness requirements to `force_close` as to all other operations
- Cross-check the payload timestamp against the last stored oracle timestamp in vAMM state before executing `force_close`
- Log `force_close` events with both the oracle price used and the current on-chain spot price for auditability

---

Finding 5 — MEDIUM: Vault Whitelist Disclosure via Public get_vault_whitelisted_addresses

Severity: Medium (Information Disclosure) **Component:** Vault contract — `get_vault_whitelisted_addresses` **Type:** Information Leakage / Attack Surface Mapping

Description

The get-method `get_vault_whitelisted_addresses` returns a complete dictionary of all privileged addresses in the Vault contract without any authentication. This is a read-only call accessible by anyone.

Whitelisted addresses likely include:

- vAMM contract addresses per market
- Admin/governance addresses
- Keeper contract addresses
- Emergency pause addresses

Impact

For an attacker, this is a complete **privilege map** of the protocol:

- Enables targeted key compromise attacks against specific high-value addresses
- Reveals the full set of addresses that, if compromised, would give control over the Vault
- Reveals the number and addresses of active vAMM markets (useful for orchestrating coordinated attacks)
- Aids in crafting precise oracle replay attacks by identifying which asset_id corresponds to which vAMM

This does not directly enable fund theft, but materially lowers the difficulty of any targeted attack.

Suggested Remediation

- If this method exists primarily for internal/debugging purposes, restrict it or remove it from production
- At minimum, document which addresses are in the whitelist and ensure each one uses hardware security (HSM, multisig) for key management
- Consider replacing the flat whitelist dict with role-based access control where each role only has the minimum necessary privileges

---

Disclosure Timeline

Date	Action
2026-05-03	Research completed, report drafted
2026-05-03	Report submitted to Storm Trade security contact
TBD	Awaiting acknowledgement (7-day SLA requested)
TBD	Public disclosure after fix or 90-day deadline

Disclosure Contact

- **Telegram:** @storm_trade_ton (official X handle, request private security channel)
- **GitHub:** https://github.com/Tsunami-Exchange — private security advisory
- **Documentation:** https://docs.storm.tg

References

- storm-contracts-specs / scheme.tlb
- storm-contracts-specs / get-methods.md
- Storm Trade Risk Management docs
- Storm Trade Keeper docs
- TON async messaging model

This report was prepared for responsible disclosure purposes only. No user assets were accessed or modified during research. All findings are based on publicly available specifications and documentation.